



SECURITY AUDIT

Symbiotic

Restaking Adapter

FINAL REPORT

June 2026

BAILSEC.IO



Disclaimer:

Security assessment projects are time-boxed and reliant on information provided by the client, its affiliates, or its partners. The findings in this report should not be considered a comprehensive list of security issues, flaws, or defects in the target system or codebase, and apply only to the specific commit hash, version, or deployment reviewed at the time of the audit. Any subsequent modification, redeployment, or upgrade is outside the scope of this report and may invalidate its findings.

The audit does not constitute investment, legal, financial, regulatory, or tax advice, nor an endorsement of any project or team. It is based on a limited review of materials provided and is delivered on an "as-is," "where-is," and "as-available" basis. Use of blockchain technology is subject to unknown risks and flaws. The scope of this assessment is technical and contract-level; unless explicitly stated otherwise, it does not cover economic or game-theoretic risks, MEV, oracle manipulation, governance attacks, centralization or admin-key risks, off-chain components, front-end code, or the behavior of third-party protocols, dependencies, libraries, or services interacted with by the audited code.

We make no warranties, express or implied, regarding the accuracy, reliability, completeness, or availability of the report or any associated services, and disclaim all implied warranties including merchantability, fitness for a particular purpose, and non-infringement. We assume no responsibility for any product, service, software, code, or material referenced by, linked to, or accessible through the report.

This report is prepared solely for the contract owner and may not be relied upon by any third party. The contract owner is responsible for making their own decisions based on the report and should seek additional professional advice as needed. To the maximum extent permitted by law, our total aggregate liability arising from or related to this audit shall not exceed the fees actually paid for the engagement. The contract owner agrees to indemnify and hold harmless the audit firm and its personnel from any claims, damages, expenses, or liabilities arising from use of or reliance on the report or the audited smart contract.

This disclaimer and any dispute arising from the audit shall be governed by the laws of Wyoming (USA), without regard to its conflict-of-laws principles. By engaging in this audit, the contract owner acknowledges and agrees to these terms.

1. Project Details

Important: Please ensure that the deployed contract matches the source-code of the last commit hash.

Project	Symbiotic – Restaking Adapter – Audit Report
Website	symbiotic.fi
Language	Solidity
Methods	Manual Analysis
Github repository	https://github.com/symbioticfi/core-mirror/tree/8d5d01a73db56b781b058870391bea19361e8a25
Resolution 1	https://github.com/symbioticfi/core-mirror/tree/f32eec02c88184af2641e07100ba1a57c734e53e

2. Detection Overview

Severity	Found	Resolved	Partially Resolved	Acknowledged (no change made)	Failed resolution	Open
High						
Medium	4	1		3		
Low	9			9		
Informational	2			2		
Governance						
Total	15	1		14		

2.1 Detection Definitions

Severity	Description
High	The problem poses a significant threat to the confidentiality of a considerable number of users' sensitive data. It also has the potential to cause severe damage to the client's reputation or result in substantial financial losses for both the client and the affected users.
Medium	While medium level vulnerabilities may not be easy to exploit, they can still have a major impact on the execution of a smart contract. For instance, they may allow public access to critical functions, which could lead to serious consequences.
Low	Poses a very low-level risk to the project or users. Nevertheless the issue should be fixed immediately
Informational	Effects are small and do not post an immediate danger to the project or users
Governance	Governance privileges which can directly result in a loss of funds or other potential undesired behavior

3. Detection

RestakingAppAdapter

`RestakingAppAdapter` is an `AppAdapter` for cases where the parent `VaultV2` asset is itself a nested `VaultV2` share token. During initialization, the adapter walks the ERC4626 asset chain from the parent vault asset down to a configured base asset and stores each vault in `underlyingVaults`. The adapter reports stake and slashable value in the configured base asset, while its direct balance accounting counts the top-level wrapped vault share, `underlyingVaults[0]`.

Rewards received or converted into the configured base asset are synchronized upward through the nested vault chain by `syncReward()`. The same function can also continue synchronization from intermediate vault-share balances held by the adapter. Slashes are requested in base-asset units, converted into top-level wrapper shares, and then unwound through each underlying vault's `WithdrawalQueue` until the final base asset can be sent to the burner.

Appendix: Nested Vault Chain

If Vault3 accepts Vault2 shares and Vault2 accepts Vault1 shares, and Vault1 accepts USDC, then a `RestakingAppAdapter` attached to Vault3 with USDC as the configured base asset stores:

`underlyingVaults[0] = Vault2`

`underlyingVaults[1] = Vault1`

`asset = USDC`

Privileged Functions

- `setConverters`
- `invalidateConvert`
- `invalidateConverts`
- `allocate`
- `deallocate`
- `requestDeallocate`

Invariants

- `underlyingVaults` must represent a valid `VaultV2/ERC4626` asset chain from the parent vault asset down to the configured base asset.
- `underlyingVaults[0]` is the direct parent-vault asset counted by `RestakingAppAdapter.totalAssets()`.
- The configured asset is the final base asset of the nested vault chain, not the direct parent-vault asset.
- `convert` must not sell the configured base asset or any vault share token in `underlyingVaults`.
- `slash` and `release` accept base-asset-denominated amounts but internally convert those amounts into top-level wrapper shares.
- Slashed wrapper shares should eventually be unwound through each underlying vault `WithdrawalQueue` and delivered as the configured base asset to the burner.

Issue_01	CowSwap convert can be sandwiched for profit
Severity	Medium
Description	<code>RestakingAppAdapter.convert()</code> only allows reward tokens to be converted into the configured base asset through CoW Swap. However, <code>RestakingAppAdapter.totalAssets()</code> only counts the adapter's balance of <code>underlyingVaults[0]</code>, the top-level wrapped vault share. This creates an accounting gap after a CoW order is filled. The bought base asset is sent to the adapter, but it is not counted by <code>totalAssets()</code> until <code>syncReward()</code> deposits it upward through the underlying vault chain and mints <code>underlyingVaults[0]</code> shares. A user can then sandwich a <code>syncReward</code> call with a deposit and a subsequent withdrawal to capture most of the profit from the cowswap conversion, denying profits to the other vault participants.
Recommendations	Consider automatically syncing after a conversion, or counting the base assets in <code>totalAssets</code> as well, to mitigate this issue partially. However, since the <code>convert</code> call will still raise <code>totalAssets</code>, the attack vector will still remain. An ideal fix would be to drip rewards over time to the vault, so that simple sandwich attacks cannot claim all the value.
Comments / Resolution	Acknowledged.

Issue_02	Restaking slashes are denominated in shares and can thus burn unpredictable amounts of assets
Severity	Medium
Description	During slashing, the amount in base asset (say USDC) is specified in the <code>RestakingAppAdapter:slash</code> call. The adapter then stores a slashing request for the corresponding number of shares. <pre><i>super.slash[_convertToShare(amount, true)];</i></pre> The issue is that while this many shares represent an amount at this point in time, this might not be true later since the vault share-asset ratio can change over time. Withdrawals from <code>AppAdapters</code> are queued and not instant, so when the withdrawals actually get filled, the actual base amount slashed can be lower or higher than the original expected amount. So a simple slash call can end up burning not enough or way more tokens, since the accounting is done in shares, whose redemption ratios are not fixed. The effect scales with vault depth. On a shallow vault, donating assets equal to TVL can roughly double the reported slashable value while the adapter still holds the same shares. That mismatch is larger than the manipulation cost when the underlying pool is small relative to the adapter position.
Recommendations	Consider redefining the slashing structure to operate on asset amounts instead of shares.
Comments / Resolution	Acknowledged.

Issue_03	Unbounded syncSlash processing can make slash settlement unable to progress
Severity	Medium
Description	<code>RestakingAppAdapter.syncSlash()</code> synchronizes pending slash withdrawal requests by iterating from <code>firstUnclaimed</code> through the stored tokenIds for each underlying vault. For every request, it reads the withdrawal queue state, checks claimable shares, attempts to claim, and then checks whether the request is fully claimed. The function has no caller-supplied batch size or maximum number of requests to process. It also updates <code>firstUnclaimed</code> only after the loop finishes. If many slash withdrawal requests accumulate while the underlying withdrawal queue is not claimable, and later many of them become claimable at once, <code>syncSlash()</code> may need to process too many external requests/claimable/claim/isClaimed calls in a single transaction. If the required work exceeds the block gas limit, the transaction reverts before <code>firstUnclaimed</code> is updated. Because no partial progress is saved, callers cannot settle the backlog in smaller batches. This can leave slashed assets stuck in withdrawal queues. The impact is larger because <code>RestakingAppAdapter.slash()</code> and <code>release()</code> both call <code>syncSlash()</code> before applying their own logic. Therefore, a large unprocessed slash backlog can also block future slash and release operations for the adapter.
Recommendations	Add bounded batch processing to <code>syncSlash()</code> , allowing callers to specify a maximum number of withdrawal requests to process and saving <code>firstUnclaimed</code> progress after each processed request or batch.
Comments / Resolution	Fixed by limiting the number of claims to <code>MAX_TOTAL_CLAIMS</code> in each <code>syncSlash</code> call.

Issue_04	Restaking App Adapter can be donated funds to, manipulating reserve limits
Severity	Low
Description	RestakingAppAdapter reports assets using the raw balance of the outermost inner-vault share token held by the adapter, not an internally tracked balance or a redemption-based value. Anyone can increase that balance without going through normal allocation limits or the guarded swap flow. This matches the donation-style accounting issue seen elsewhere in the codebase. Adapter limits can stop new capital from entering through allocate, but they do not stop someone from pushing value into reported NAV through reward or a direct transfer. On withdrawal, the delegator deallocates from adapters in order and pulls whatever is liquid. Inflated restaking exposure stays in the adapter while redeemers are paid from more liquid routes, leaving remaining depositors with shares backed partly by assets they cannot easily exit.
Recommendations	Consider using storage-based accounting instead.
Comments / Resolution	Acknowledged.

Issue_05	Partial reward synchronization can strand intermediate vault shares outside adapter accounting
Severity	Low
Description	<code>RestakingAppAdapter.syncReward()</code> synchronizes rewards by depositing held assets upward through the configured nested vault chain. It starts from the configured base asset and, for each layer, deposits the adapter's current balance of the asset accepted by that vault. However, <code>syncReward()</code> does not check whether the full nested deposit path can be completed before starting the first deposit. For each vault, it only deposits up to <code>min(balance, maxDeposit(address[this]))</code>. If an inner deposit succeeds but a later outer vault cannot accept the full resulting balance, the adapter can be left holding intermediate vault shares. For example, in a Vault2 -> Vault1 -> USDC chain, <code>syncReward()</code> may deposit USDC into Vault1 and receive Vault1 shares. If Vault2 then has <code>maxDeposit(address[this])</code> equal to zero, or lower than the adapter's Vault1 share balance, those Vault1 shares remain held by the adapter. The problem is that <code>RestakingAppAdapter.totalAssets()</code> only counts the adapter's balance of <code>underlyingVaults[0]</code>, the top-level wrapped vault share. Intermediate vault-share balances are not counted. As a result, after a partial <code>syncReward()</code>, the adapter can hold real reward value that remains invisible to parent vault/delegator accounting until the outer vault later accepts deposits and <code>syncReward()</code> is called again. This can temporarily underreport vault value and allow users to enter or exit while part of the adapter's reward value is stuck at an intermediate layer outside <code>totalAssets()</code>, leading to a similar sandwich attack as discussed above in a different issue.
Recommendations	Consider including intermediate vault-share balances in <code>totalAssets()</code> , or making <code>syncReward()</code> check the full nested deposit path before depositing into lower layers.

Comments / Resolution	Acknowledged.
--------------------------	---------------

Issue_06	Unsynced converted rewards remain outside restaking slash accounting
Severity	Low
Description	<code>RestakingAppAdapter</code> slash accounting is performed against the currently accounted top-level wrapper shares. <code>RestakingAppAdapter.totalAssets()</code> only counts <code>underlyingVaults[0]</code> shares held by the adapter, while raw configured base assets held by the adapter are ignored until <code>syncReward()</code> deposits them through the nested vault chain. This creates a slash accounting mismatch when CoW-converted rewards are sitting in the adapter as the configured base asset. If a slash happens before <code>syncReward()</code> is called, the slash is applied only to the currently accounted wrapper shares. The raw base assets remain in the adapter and are not part of the wrapper-share amount being sent through the withdrawal queues. After the slash, anyone can call <code>syncReward()</code>. The previously unsynced base assets are then deposited through the underlying vault chain and minted into new <code>underlyingVaults[0]</code> shares. This means the value that was already held by the adapter at the time of the slash was not considered by the restaking slash flow because it had not yet been synchronized into the counted wrapper-share balance. This can cause slash execution and adapter accounting to observe different asset sets during the unsynced window. The impact depends on whether converted rewards are intended to be included in the slashable restaking value immediately or only after synchronization.
Recommendations	Synchronize or explicitly account for all base-asset and intermediate reward balances before applying restaking slash accounting, or document that unsynced reward balances are outside the slashable restaking position until <code>syncReward()</code> runs.
Comments / Resolution	Acknowledged.

Issue_07	Over-bound restaking slashes can revert before being capped to slashable stake
Severity	Low
Description	<code>RestakingAppAdapter.slash[amount]</code> accepts an amount denominated in the configured base asset. The interface documents this value as the maximum amount to slash, and the base <code>AppAdapter.slash()</code> implementation normally caps the requested amount to the current slashable stake before applying the slash. <code>RestakingAppAdapter</code> changes this order. It first converts the caller-supplied base-asset amount into top-level wrapper shares using <code>_convertToShare[amount, true]</code>, and only then calls the base <code>AppAdapter.slash()</code> logic. As a result, very large over-bound amounts can revert during the <code>ERC4626 previewWithdraw</code> conversion before <code>AppAdapter.slash()</code> has a chance to cap the amount to the actually available slashable stake. For example, an integration may pass <code>slash[type(uint256).max]</code>, expecting the adapter to slash all currently available stake. In <code>RestakingAppAdapter</code>, that call can revert during the nested share conversion, while passing the exact current slashable amount would succeed.
Recommendations	Cap the requested base-asset amount to the current base-asset-denominated slashable value before converting it into wrapper shares, or explicitly require callers to pass exact bounded restaking slash amounts.
Comments / Resolution	Acknowledged.

Issue_08	Restaking release can leave dust slashable stake due to round-down share conversion
Severity	Low
Description	<code>RestakingAppAdapter.release(amount)</code> accepts an amount denominated in the configured base asset, while the base AppAdapter release accounting is performed in top-level wrapper shares. <code>RestakingAppAdapter.slashable()</code> reports the current slashable value by converting wrapper shares into the configured base asset through <code>previewRedeem()</code>. When <code>release(adapter.slashable())</code> is called, the adapter converts that base-asset amount back into wrapper shares using <code>_convertToShare(amount, false)</code>, which uses <code>previewDeposit()</code> at each layer. This conversion rounds down. In fractional share-price cases, converting from shares to assets and then back from assets to shares can return fewer wrapper shares than the original slashable position. As a result, a caller attempting to release the exact amount returned by <code>slashable()</code> can leave a small amount of slashable wrapper shares behind. In extreme dust cases, the converted share amount can be zero. The remaining amount can only be cleared by passing an over-bound amount. The impact is bounded by rounding, but it breaks the expected behavior that releasing the reported slashable amount clears the reported slashable position.
Recommendations	When releasing the full reported slashable amount, cap or round the conversion so the corresponding full wrapper-share position is cleared, or expose a share-denominated release path for restaking adapters.
Comments / Resolution	Acknowledged.

Issue_09	syncReward can donate reward assets when an underlying vault mints zero shares
Severity	Low
Description	<code>RestakingAppAdapter.syncReward()</code> deposits held base or intermediate assets upward through the configured ERC4626 vault chain. For each layer, it calls <code>deposit[amount, address[this]]</code> but does not check how many shares were minted. <code>VaultV2/ERC4626 deposit()</code> can return zero shares for a non-zero asset amount due to floor rounding, dust-sized deposits, or a very high share price. If this happens, the adapter transfers reward assets into the underlying vault but receives no useful accounting shares in return. Those assets are effectively donated to the underlying vault's existing shareholders instead of becoming counted assets of the <code>RestakingAppAdapter</code>. This is especially relevant because <code>RestakingAppAdapter.totalAssets()</code> only counts the final top-level wrapper share balance, so a zero-share deposit gives the adapter no balance that can later be counted or recovered through normal accounting. The impact is limited to dust or rounding-bounded amounts in normal configurations, but the adapter should not silently donate reward assets during synchronization.
Recommendations	Check the shares returned by each <code>deposit()</code> in <code>syncReward()</code> and skip or revert when a non-zero asset deposit would mint zero shares.
Comments / Resolution	Acknowledged.

Issue_10	Reward delivery fails when underlying vault deposit reverts
Severity	Informational
Description	In <code>RestakingAppAdapter</code>, <code>reward()</code> pulls base asset from the caller and then immediately calls <code>syncReward()</code>, which deposits that balance into the underlying <code>ERC4626</code> vault chain. There is no error handling around those deposits. If any underlying vault reverts on <code>deposit()</code> – for example, because it is paused, has hit a deposit cap, or enforces access control – the entire <code>reward()</code> transaction reverts, and the adapter never receives that reward. The main risk is a missed reward. Many distributors only attempt delivery once: an epoch payout, a one-time airdrop, or a Merkl-style claim. If that single call fails, the adapter may never get those rewards unless the distributor retries after the vault accepts deposits again. The adapter has no control over when or whether that happens.
Recommendations	Make sure any contract calling reward is capable of handling failure modes.
Comments / Resolution	Acknowledged.

Issue_11	Broken underlying vault permanently strands slash proceeds in the restaking chain
Severity	Informational
Description	<code>RestakingAppAdapter</code> unwinds slashed exposure by redeeming through a chain of underlying <code>ERC4626</code> vaults and their withdrawal queues. Each hop claims assets from one vault's queue and immediately opens a redeem request in the next vault downstream. The process assumes every vault in the chain can still convert shares back toward the configured base asset. That assumption breaks when any intermediate vault is economically dead: <code>totalAssets</code> is zero (or near zero) while <code>totalSupply</code> remains positive. This can happen after a severe slash on the underlying restaking vault. The vault's share accounting still works – it accepts withdrawal requests and processes claims in share units – but every redeem returns zero or negligible assets. In such a case, funds can get stranded in the system, unable to be retrieved.
Recommendations	Consider adding a path to remove dead vault from the chain, and admin function for withdrawing shares for vaults in the chain.
Comments / Resolution	Acknowledged.

AaveV3Adapter

AaveV3Adapter is a VaultV2 adapter for Aave V3 supply positions. The adapter supplies the parent **VaultV2** asset into the configured Aave V3 pool and accounts for the resulting **aToken** balance as deployed assets. Deallocation withdraws from Aave up to the adapter's **aToken** balance and currently available Aave liquidity.

This review is **differential against** the previously audited adapter version. The relevant new surface is the addition of **MerklClaimer** and **CoWSwapConverter**. **MerklClaimer** allows **Merkl** rewards to be claimed into the adapter. **CoWSwapConverter** allows non-vault-asset, non-aToken reward tokens held by the adapter to be asynchronously converted through CoW Swap into the parent vault asset.

Appendix: Conversion Flow

The adapter rejects conversions where tokenIn is the parent vault asset or the configured aToken. The buy token must be the parent VaultV2 asset. Calling convert does not synchronously swap the reward token. It pre-signs a partially fillable CoW order with the adapter as receiver, so the reward token remains in the adapter until CoW settlement fills the order and sends vault assets back to the adapter.

Privileged Functions

- setConverters
- invalidateConvert
- invalidateConverts
- allocate
- deallocate
- requestDeallocate

Invariants

- The adapter must bind to the Aave reserve **aToken** for the parent **VaultV2** asset during initialization and must not account for any other **aToken** as deployed assets.
- Adapter accounting must include free parent-vault assets and the adapter's configured **aToken** balance.
- The adapter must not create CoW orders that sell the parent **VaultV2** asset or the configured aToken.
- All CoW conversion orders must buy the parent **VaultV2** asset and use the adapter as receiver.

Issue_12	Merkle claims and converts can be sandwiched for profit
Severity	Medium
Description	Both AaveV3Adapter and MorphoV2Adapter only count their internal token and vault asset in totalAssets, not taking into consideration any pending Merkl rewards. The convert function converts uncounted assets into the vault asset, which is counted, essentially increasing the value of the vault in a single transaction. This allows any user to deposit a large amount of funds into the vault, trigger the merkle reward claim and conversion, and then withdraw to claim a large part of the merkl rewards. Thus, even when the Adapters have claims on Merkl or outstanding cswap orders, it will underreport its own assets, allowing any user to capture most of the profit.
Recommendations	Consider accounting for the rewards in totalAssets as well, or drip the rewards over time to the Adapter, eliminating the sandwich vector.
Comments / Resolution	Acknowledged.

Issue_13	Open convert function allows any swap price
Severity	Low
Description	The <code>convert</code> functions in all the adapters allow anyone to call them to trigger a cowswap swap. In <code>CoWSwapConverter</code>, callers must queue a swap transaction first, which can only execute after a set period of time. Firstly, the <code>prepareConvert</code> function in <code>CoWSwapConverter</code> is open and allows any user to set any price, which can lead to bad swaps. Furthermore, the whole swap queue system is skipped if the adapter itself is also designated as a <code>converter</code>. In that case, any user can carry out a swap at any price at a loss to the protocol.
Recommendations	Consider gating the <code>prepareConvert</code> function or having minimum price checks. Also, do not set the adapter itself as a <code>converter</code> , since that can lead to the same issue.
Comments / Resolution	Acknowledged.

Issue_14	Merkle claims can let late depositors share previously accrued rewards
Severity	Low
Description	Both <code>AaveV3Adapter</code> and <code>MorphoVaultV2Adapter</code> inherit <code>MerkleClaimer</code>, which allows anyone to claim Merkle rewards into the adapter. However, claimable Merkle rewards are not reflected in <code>totalAssets()</code> before they are claimed. <code>totalAssets()</code> only accounts for free parent-vault assets and the adapter's configured asset. Therefore, rewards that have accrued to the adapter through <code>Merkle</code> remain outside vault accounting until <code>claim()</code> is called. This creates a timing window where a user can deposit into the parent vault before triggering a Merkle claim. After the deposit, the user can call <code>claim()</code>, causing previously accrued rewards to be received by the adapter. If the claimed token is the parent vault asset, it is counted as free assets. If the claimed token is another reward token, it remains outside accounting until converted through the adapter's CoW conversion path. As a result, a late depositor can share the reward value that was accrued before their deposit. The impact depends on whether unclaimed Merkle rewards are intended to be treated as existing vault value before they are claimed.
Recommendations	Define whether unclaimed Merkle rewards are part of the existing vault value. If they are, include claimable rewards in accounting or restrict/sequence claim operations so deposits cannot occur immediately before claiming previously accrued rewards.
Comments / Resolution	Acknowledged.

MorphoVaultV2Adapter

MorphoVaultV2Adapter is a **VaultV2** adapter for Morpho Vault V2 positions. The adapter deposits the parent **VaultV2** asset into the configured Morpho vault and tracks the deployed position through the internal **totalShares** value. Adapter accounting reports free parent-vault assets plus the parent-asset value represented by **totalShares**.

This review is **differential against** the previously audited adapter version. The relevant new surface is the addition of **MerkIClaimer** and **CoWSwapConverter**. **MerkIClaimer** allows **MerkI** rewards to be claimed into the adapter. **CoWSwapConverter** allows non-vault-asset, non-morphoVault reward tokens held by the adapter to be asynchronously converted through CoW Swap into the parent vault asset.

Appendix: Conversion Flow

The adapter rejects conversions where **tokenIn** is the parent vault asset or the configured **morphoVault** share token. The buy token must be the parent **VaultV2** asset. Calling **convert** does not synchronously swap the reward token. It pre-signs a partially fillable CoW order with the adapter as receiver, so the reward token remains in the adapter until CoW settlement fills the order and sends parent-vault assets back to the adapter.

Privileged Functions

- **setConverters**
- **invalidateConvert**
- **invalidateConverts**
- **allocate**
- **deallocate**
- **requestDeallocate**

Invariants

- The configured **morphoVault** must be a valid Morpho Vault V2 for the parent **VaultV2** asset.
- Adapter accounting must include free parent-vault assets and the assets represented by tracked **totalShares**.
- **totalShares** must increase only when the adapter successfully deposits into the configured Morpho vault.
- **totalShares** must decrease when tracked Morpho shares are withdrawn during deallocation.
- The adapter must not create CoW orders that sell the parent **VaultV2** asset or the configured **morphoVault** share token.
- All CoW conversion orders must buy the parent **VaultV2** asset and use the adapter as the receiver.

Issue_15	Claimed Morpho vault shares can remain unaccounted for and stuck
Severity	Low
Description	<code>MorphoVaultV2Adapter</code> tracks its deployed Morpho position through the internal <code>totalShares</code> value. <code>totalAssets()</code> does not read the adapter's actual <code>morphoVault</code> balance directly. Instead, it reports free parent-vault assets plus <code>morphoVault.previewRedeem(totalShares)</code>. The new Merkl claim path can transfer reward tokens into the adapter without updating <code>totalShares</code>. If the claimed reward token is the configured <code>morphoVault</code> share token itself, the adapter receives additional Morpho vault shares, but those shares are not added to <code>totalShares</code>. As a result, the claimed Morpho vault shares are not counted by <code>totalAssets()</code>. They also cannot be converted through CoW Swap because <code>convert()</code> rejects tokenIn equal to the configured <code>morphoVault</code>. When deallocation occurs, it is capped by the assets represented by tracked <code>totalShares</code>, so the untracked claimed shares can remain in the adapter after all tracked shares have been withdrawn. This can leave claimed Morpho vault shares unaccounted for and stuck in the adapter.
Recommendations	If <code>morphoVault</code> shares can be claimed as rewards, either reject such claims, add claimed <code>morphoVault</code> shares to <code>totalShares</code>, or provide a recovery/accounting path for untracked Morpho vault share balances. Otherwise, consider documenting and acknowledging this issue.
Comments / Resolution	Acknowledged.